

The **lion-antelope** pursuit problem from Tutorial 2 is based on two simplifying assumptions:

- The **lion** chases at a constant speed, always running directly toward the **antelope**.
- The **antelope** flees at constant speed, always running in a straight line.

In the wild, predators use much more efficient strategies for catching their prey: For example, the lion might steer towards a predicted interception point instead of aiming at the instantaneous position of the antelope. In this coding challenge we will investigate such strategies using a generalization of the model from Tutorial 2.

As you work through this coding challenge, you will become familiar with:

- Using Python to numerically solve differential equations.
- Converting the rules of the pursuit game into a system of ODEs.
- Visualizing numerical solutions to refine your own strategy for the game.

Mathematical model. At each time t , the state of each animal is given by its **position** in the xy -plane and **direction of travel** $\phi(t)$ measured counter-clockwise from the positive x -axis. Using the notation $(x_L(t), y_L(t), \phi_L(t))$ for the lion and $(x_A(t), y_A(t), \phi_A(t))$ for the antelope, these variables evolve as follows:

$$\frac{d}{dt}x_L(t) = v_L \cos \phi_L(t), \quad \frac{d}{dt}y_L(t) = v_L \sin \phi_L(t), \quad \frac{d}{dt}\phi_L(t) = J_L(\text{state}(t)), \quad (1)$$

$$\frac{d}{dt}x_A(t) = v_A \cos \phi_A(t), \quad \frac{d}{dt}y_A(t) = v_A \sin \phi_A(t), \quad \frac{d}{dt}\phi_A(t) = J_A(\text{state}(t)). \quad (2)$$

where parameters v_L and v_A are the **fixed speed** of each animal, the **steering functions** $J_L(\text{state}(t))$ and $J_A(\text{state}(t))$ describe the rate at which each animal changes its direction of travel over time, and

$$\text{state}(t) = (x_L(t), y_L(t), \phi_L(t), x_A(t), y_A(t), \phi_A(t))$$

is the **state** of the entire system at time t , see Figure 1.

Assumptions. We assume the animals have limited abilities and unique advantages:

- Both animals have constant speed (i.e. they can only change the direction in which they run)
- The **lion** is faster than the **antelope** ($v_L > v_A > 0$).
- Both animals have finite agility; they cannot turn arbitrarily quickly ($|J_A(\text{state}(t))| \leq J_A^{\max}$, $|J_L(\text{state}(t))| \leq J_L^{\max}$).
- The **antelope** is more agile than the **lion** ($J_A^{\max} > J_L^{\max} > 0$).
- The animals are aware of each other's position and orientation and can use this information strategically.
- The **lion** wants to catch the **antelope** as quickly as possible.
- The **antelope** wants to evade capture for as long as possible.

Conventions. We will always use a single vector to represent the state of the system at each point in time. We follow the convention that this vector is of the form $\mathbf{x} = [x_L, y_L, \phi_L, x_A, y_A, \phi_A]$, where

- $[x_L, y_L, \phi_L]$ are the **lion's** cartesian coordinates and orientation,
- $[x_A, y_A, \phi_A]$ are the **antelope's** cartesian coordinates and orientation.

We use the terms orientation and direction of travel interchangeably.

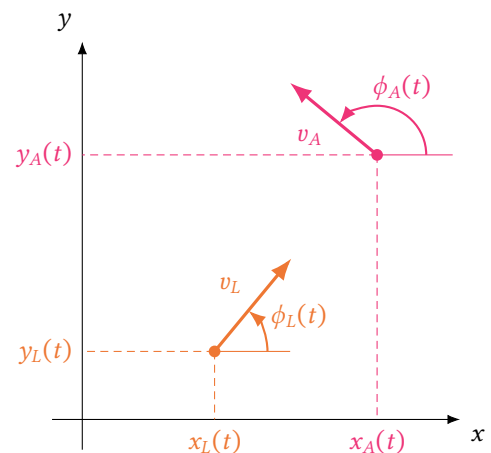


Figure 1: State variables and speed parameters in the **lion-antelope** model.

Q1: Simulate the model (3 visible points)

Numerical simulation of ODE models typically follows this basic structure: implement the right-hand side of the ODE as a callable function, specify the parameters and initial conditions, and solve the system using a suitable numerical method. We haven't learned about numerical methods to solve ODEs yet, so for now we will use a standard ODE solver available in SciPy called `solve_ivp`.

(a) Let's solve the model numerically and visualize the result. Consider the following initial conditions

$$\begin{aligned}(x_L(0), y_L(0)) &= (0, 0), & \phi_L(0) &= \frac{\pi}{2}, \\(x_A(0), y_A(0)) &= (1, 0), & \phi_A(0) &= -\frac{\pi}{2},\end{aligned}$$

and suppose $v_L = 2$, $v_A = 1$, and the steering functions are given by

$$\begin{aligned}\frac{d}{dt}\phi_L(t) &= J_L(t, \text{state}(t)) = -t, \\ \frac{d}{dt}\phi_A(t) &= J_A(t, \text{state}(t)) = 1.\end{aligned}$$

These functions can be summarized as: The antelope turns counterclockwise at a fixed rate. The lion turns clockwise, but turns faster and faster. These steering functions are absolutely not a good pursuit strategy. In particular neither the antelope nor the lion seem to care at all where the other one is, they just keep turning. Also, J_L is unbounded which breaks our agility restriction. We just use them as a first example to illustrate the basics of numerical simulation.

In `q1.py`, implement the functions $J_L(t, x)$ and $J_A(t, x)$ to match the functions above.

(b) `q1.py` defines a function `lion_ante_rhs(t, x, vL, vA, JL, JA)` to evaluate the right-hand side of the equations (1) and (2) on the first page. It receives the following inputs:

- `x` is the current state `[xL, yL, phiL, xA, yA, phiA]`,
- `t` is the current time,
- `vL` and `vA` are the fixed lion and antelope speeds,
- `JL` is the lion steering function, callable as `JL(t, x)`
- `JA` is the antelope steering function, callable as `JA(t, x)`,

Implement `lion_ante_rhs` so that it returns the derivative vector

`dx = [dxL, dyL, dphiL, dxA, dyA, dphiA]`.

The rest of `q1.py` includes code for visualizing your solution. It already contains the parameters, initial state, imports and function headers. It also includes the code to numerically solve the IVP using `solve_ivp` and to then plot the solutions. The most important lines are:

```
1 solution = solve_ivp(
2     lambda t, x: lion_ante_rhs(t, x, vL, vA, JL, JA),
3     [0, 5], x0, t_eval=times, rtol=1e-8, atol=1e-10
4 )
5 [...]
6 x_final = solution.y[:, -1]
7 [...]
8 utils.plot_trajectories(solution, "q1_plot.png")
```

The parameters `rtol` and `atol` are the relative and absolute error tolerances that the ODE solver uses to choose its step sizes adaptively. The relative tolerance controls how many significant digits are accurate when solution values are large, while the absolute tolerance sets a floor on the accuracy demanded when values are near zero, so the solver does not stall on negligible quantities.

Q2: Implement realistic strategies (3 visible points)

Instead of the “crazy” ones in Q1, we now implement sensible strategies for each animal. Both strategies are based on a **clip-wrap steering function**: the animal’s current direction of travel $\phi(t)$ is pushed toward a target angle θ_{target} at a rate that is proportional to their difference. To ensure that the steering obeys the agility constraints, we clip the steering function so that it stays constant once it has reached the animal’s maximum steering rate, see Fig. 2. The clip-wrap steering function J_{cw} can be stated explicitly

$$\frac{d}{dt}\phi(t) = J_{\text{cw}}(\phi(t); \theta_{\text{target}}, J^{\text{max}}) = \text{clip}(\text{wrap}(\theta_{\text{target}} - \phi(t)), J^{\text{max}}), \quad (3)$$

where the clip and wrap functions are given by

$$\text{wrap}(\alpha) = ((\alpha + \pi) \bmod 2\pi) - \pi, \quad \text{clip}(x, J^{\text{max}}) = \begin{cases} -J^{\text{max}}, & x < -J^{\text{max}}, \\ x, & -J^{\text{max}} \leq x \leq J^{\text{max}}, \\ J^{\text{max}}, & x > J^{\text{max}}. \end{cases} \quad (4)$$

We set the agility constraints to be $J_L^{\text{max}} = 1$, $J_A^{\text{max}} = 2$ and the speed of the animals to be $v_L = 1.5$, $v_A = 1$.

- (a) Implement `J_cw_lion(t,x)` in `q2.py`. Its target angle points from the **lion** to the **antelope**. With `xL`, `yL`, `phiL`, `xA`, `yA`, `phiA` = `x`, Python computes that angle as

```
theta = np.arctan2(yA-yL, xA-xL)
```

Complete the clip-wrap calculation using Eqs. (3)-(4), with turning limit `JLmax`.

- (b) Implement `J_cw_ante(t,x)` in `q2.py`. Its target angle is 90 degrees counterclockwise relative to the line from **lion** to **antelope**:

```
theta = np.arctan2(yA-yL, xA-xL) + np.pi/2
```

Complete the clip-wrap calculation with turning limit `JAmay`.

- (c) For our simulation, we will use the initial state

$$\mathbf{x}_0 = [0, 0, 0, 3, 0, \pi/2].$$

Thus the **lion** starts at $(0, 0)$ pointing at the **antelope** at $(3, 0)$, and the **antelope** initially points upward. The script `q2.py` already contains the code to compute the position of both animals at time $t = 6$ and to plot their trajectories. The number of evaluations and the error bounds are the same as in Q1.

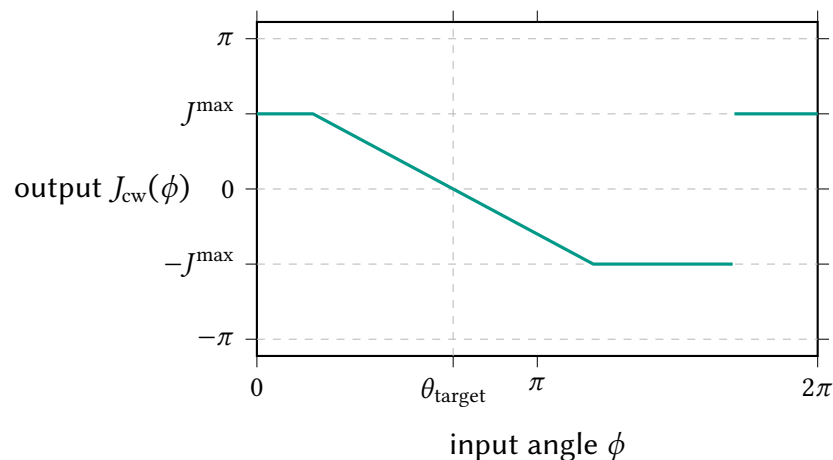


Figure 2: The clip-wrap steering function $J_{\text{cw}}(\phi; \theta_{\text{target}}, J^{\text{max}}) = \text{clip}(\text{wrap}(\theta_{\text{target}} - \phi), J^{\text{max}})$

Q3: Compute survival time (3 hidden points)

We now add **event-tracking**. The simulation should stop at a time t_0 where the **lion** and **antelope** are at the same position

$$x_L(t_0) = x_A(t_0) \quad \text{and} \quad y_L(t_0) = y_A(t_0), \quad (5)$$

meaning that the **antelope** has been caught. We refer to the first such time t_0 satisfying Eq. (5) as the **survival time** of the **antelope**. Since we are solving the ODE numerically, we cannot use Eq. (5) directly (we should never expect two approximate solutions to be *exactly* equal in floating point). We will instead seek to identify the first time t^* where the **lion** and **antelope** are at effectively the same position:

$$\sqrt{|x_L(t^*) - x_A(t^*)|^2 + |y_L(t^*) - y_A(t^*)|^2} \leq r_{\text{collision}} \quad (6)$$

where $r_{\text{collision}} = 0.05$ is the prescribed collision radius. We refer to the first t^* satisfying Eq. (6) as the **approximate survival time** of the **antelope**. If no capture is detected by the final time, we want to report that the survival time exceeds the simulated horizon; in particular, it should not report the final time as a capture time.

- (a) Implement event tracking using the starter code in q3.py. The script already imports the steering functions from q2.py with turning limits $J_L^{\max} = 1$ and $J_A^{\max} = 2$. It uses speeds $v_L = 2$ and $v_A = 0.75$, with $T_{\max}=6$ and initial state

$$x_0 = [0.75, 0, \pi/4, 3, 0, \pi/2].$$

Both animals start on $y = 0$, separated by 2.25 units. The **antelope** initially runs upward, while the **lion** initially heads at 45° toward its path. The file uses $r_{\text{collision}} = 0.05$ and $\text{max_step} = 5 \times 10^{-4}$ with the default solver tolerances. Your event function should return the separation between the animals minus $r_{\text{collision}}$ (i.e., the difference between the left- and right-hand sides of Eq. (6)). Thus, the event function is positive while the animals are outside the capture radius, zero at capture, and negative inside it.

The event function is also assigned `terminal = True`. This means that the solver stops at (approximate) capture. Without this setting, the solver would only record the collision time but then keep going.

- (b) Run the simulation with the Q3 parameters and initial state above, using event tracking to determine the approximate survival time. The essential addition to the usual `solve_ivp` call is `events=collision_event`:

```
1 solution = solve_ivp(
2     lambda t, x: lion_ante_rhs(t, x, vL, vA, J_cw_lion, J_cw_ante),
3     [0, Tmax], x0, events=collision_event,
4     max_step=max_step, dense_output=True
5 )
6 if len(solution.t_events[0]) > 0:
7     survival_time = solution.t_events[0][0]
8 else:
9     survival_time = None
```

q3.py already includes the code to plot the trajectories up to capture, or to the search horizon in case no capture occurs.

Q4: Challenge (3 visible points)

For the challenge, you will create a strategy for either a **lion** or an **antelope**. The best lion and the best antelope will win the coding challenge.

Determine which side you are on. If you are completing this assignment...

...by yourself, then $n = [\text{last digit of your student number}]$

...with a partner, then $n = [\text{last digit of your student number}] + [\text{last digit of your partner's number}]$

If n is even, then you are “Team Lion”. If n is odd, then you are “Team Antelope”.

For example, if your student numbers end in 7 and 5, then $7 + 5 = 12$ is even, so you are “Team Lion”.

Devise a strategy. Submit one steering function $J_strategy(t, x)$ for the animal's steering rate. The output must be finite and may depend on the current time t and the current state x .

In `challenge.py`, set `TEAM` and `TEAMNAME`, and define the `J_strategy` function. Everything else is already included.

The simulation uses the following setup:

$$v_L = 2 \quad v_A = \frac{1}{1 + t^2} \quad \text{initial state } x_0 = [0, 0, \pi/6, 2.25, 0, 0] \quad r_{\text{collision}} = 0.05 \quad \text{maximum time } T_{\text{max}} = 10$$

Note, in particular, that the antelope is getting slower and slower because it gets tired.

Complete submission. To receive the full grade, your submission must fulfil the following conditions:

If you are “Team Lion”:

- $|J| \leq J_L^{\text{max}} = 1$ must be true no matter the input.
- Compare these two races:
 - Clip-Wrap-Lion versus Clip-Wrap-Antelope
 - Your lion versus Clip-Wrap-AntelopeYour lion must catch the antelope faster.

If you are “Team Antelope”:

- $|J| \leq J_A^{\text{max}} = 2$ must be true no matter the input.
- Compare these two races:
 - Clip-Wrap-Lion versus Clip-Wrap-Antelope
 - Clip-Wrap-Lion versus your antelopeYour antelope must survive longer.

Additionally, your strategy must use the current position of the lion and the antelope. For example, a strategy like “always run in a circle” is not accepted, since it completely ignores the actual situation.

When you submit, the auto-grader will confirm that all of these requirements are met.

The competition. After everyone has submitted their assignment, we will simulate a race between each lion and each antelope.

The best lion is the one with the smallest average time to catch all the antelopes.

If there is an antelope that the lion can't catch within T_{max} , then it counts as if it did catch it at T_{max} .

The best antelope is the one with the longest average time to survive all the lions.

If there is a lion that can't catch the antelope within T_{max} , then it counts as if it was caught at T_{max} .

The winners will receive a MAT292 novelty mug and recognition in class.